
COMP8037 - Major Project Proposal

QueryLens: A Python-Based Tool for SQL Server Query and Stored Procedure Performance Optimization

Student Name: Parshotam Parmar

Date: Nov 24, 2025

Table of Contents

Introduction.....	2
The Story.....	2
Executive Summary.....	3
Biography.....	4
Introduction and Background.....	4
Goals and Proposed System.....	5
System Architecture and Design.....	5
Scope.....	8
Success Metrics.....	9
Innovation and Differentiation.....	10
Risks and Mitigations.....	10
Testing & Validation Plans.....	10
Schedule and Milestones.....	12
Assumptions and Constraints.....	13
Ethics and Data Governance.....	13
Feasibility and Build Plan.....	13
Canonical Terms.....	13
Related Work.....	14
References.....	14

Introduction

In modern database-driven applications, even small inefficiencies in SQL queries can ripple into significant performance problems under real-world load. When dozens or hundreds of users execute transactions or reports concurrently, a single poorly written query can slow an entire system and negatively affect user experience. Common issues such as retrieving unnecessary columns, applying filters that prevent index usage, or relying on implicit data type conversions often remain hidden during development. Although Microsoft SQL Server provides detailed execution plans that reveal how queries are processed internally, interpreting these plans requires specialized expertise that many developers and analysts do not possess. As a result, performance issues frequently persist in production environments until they reach a critical level.

This project proposes QueryLens, a Python-based tool designed to address this gap by making SQL Server performance analysis accessible, consistent, and automated. QueryLens will identify inefficient SQL Server queries and stored procedures by combining static code inspection with execution plan analysis, producing clear, plain-language performance recommendations. By translating low-level execution plan data into practical guidance, QueryLens is intended to support developers and analysts who work with SQL Server but may lack deep tuning expertise. Through this approach, the system aims to improve database performance proactively rather than reactively, reducing the risk of user-visible slowdowns in high-volume transactional systems.

The Story

In many business environments, software performance is judged not by benchmarks but by how smoothly everyday work gets done. When transactions take longer than expected, reports lag, or systems freeze during busy periods, users often experience frustration without knowing the technical cause. These slowdowns can delay customer service, disrupt operations, and reduce confidence in the system—even when the underlying hardware and software appear to be functioning normally.

Behind many of these issues are small inefficiencies in how databases are queried. These problems are difficult to detect early because they often remain hidden during development and only appear under real workload conditions. Developers are typically expected to fix them after users are already affected. QueryLens is motivated by this gap between when performance problems begin and when they are finally understood.

By making database performance issues visible and explainable early in the development and testing process, QueryLens aims to shift performance tuning from a reactive task to a proactive practice that benefits both developers and end users.

Executive Summary

In database-driven applications, system performance is strongly influenced by how efficiently SQL queries are written and executed. As transaction volumes increase, even minor inefficiencies can accumulate into significant slowdowns that affect both users and business operations. Common problems such as unnecessary data retrieval, mismatched data type comparisons, and query conditions that prevent index usage are difficult to detect through casual inspection. While SQL Server provides detailed execution plans that expose internal query behavior, interpreting these plans requires specialized knowledge that many development teams do not consistently have available.

QueryLens is a Python-based diagnostic tool that is designed to address this challenge by automatically detecting and explaining SQL Server query inefficiencies. The system will unify two complementary techniques: static code analysis, which examines SQL text for known inefficient patterns, and runtime analysis, which evaluates SQL Server execution plans to confirm actual performance impact. By combining these two perspectives, QueryLens is intended to produce accurate, trustworthy, and actionable performance recommendations that help teams improve database performance before inefficiencies escalate into production issues.

The system will parse SQL queries, extract execution plans, and produce plain-language reports highlighting optimization opportunities—such as rewriting non-sargable conditions, reducing unnecessary column retrieval, or improving join performance. Built with Python's *pyodbc* for database connectivity and *ANTLR* for SQL parsing, the tool will interpret execution plan XMLs to detect costly operations and missing indexes. The completed prototype will operate exclusively on SQL Server, ensuring depth and accuracy within one platform.

The project will be completed over approximately **360 hours**, divided across environment setup, static analyzer development, execution plan parsing, result integration, and testing. Success will be defined by the tool's ability to detect inefficiencies, connect them with runtime behavior, and generate readable, trustworthy recommendations. QueryLens directly addresses real-world challenges faced by database teams and demonstrates how applied data analysis and automation can improve the performance and reliability of enterprise systems. If the core system is

completed ahead of schedule, an exploratory investigation into automated query rewriting may be conducted using the existing abstract syntax tree representation, without expanding the project's required scope.

Biography

My background in database systems and software development directly supports this project. I completed the Computer Systems Technology Diploma in the Information Systems option and am currently pursuing the Bachelor of Technology in Computer Systems, specializing in the Database option. These programs built my foundation in relational database design, query optimization, and back-end integration.

Professionally, I have worked for twelve years at a golf point-of-sale software company that maintains multiple SQL Server-based systems. In my current role as a Support Data Analyst, I investigate performance issues in production databases daily. I have learned how small inefficiencies—like poor indexing or unnecessary data retrieval—can cascade into user-visible slowdowns. QueryLens grows directly from these experiences, turning practical insight into an automated tool that helps others find and fix similar problems quickly.

Although I have not previously used ANTLR directly, I am comfortable working with grammar-based parsers and structured representations of code from my coursework in database systems, compiler concepts, and backend development. I have already reviewed the T-SQL grammar provided in the ANTLR grammars-v4 repository and confirmed that it aligns with the subset of SQL required for this project. My experience working with complex stored procedures and query structures gives me a strong foundation for interpreting the abstract syntax trees that ANTLR produces. To minimize risk, I have allocated time in the early development phase to complete small prototype exercises using ANTLR's Python runtime—specifically lexical parsing, tree walking, and node extraction—before integrating it into QueryLens. With this preparation and the structured grammar provided, I will be prepared to use ANTLR effectively within the scope and timeline of this project.

Introduction and Background

Database performance remains one of the most critical factors in user satisfaction for modern applications. A well-designed system can still experience lag if its queries are inefficient. Common problems include retrieving more columns than necessary, filtering data in ways that prevent index use, or relying on implicit type conversions. SQL

Server's execution plans contain all the information needed to diagnose such issues, but interpreting them demands advanced expertise that many developers lack.

As applications scale, this skill gap creates a barrier: developers write queries, database administrators fix them later, and inefficiencies remain hidden until load increases. QueryLens bridges that gap by automating both the detection and explanation of SQL performance problems.

Goals and Proposed System

QueryLens will analyze SQL Server queries in two complementary ways:

1. **Static Analysis:** Reading SQL text to detect patterns known to reduce performance.
2. **Runtime Validation:** Reviewing the database engine's execution plan to confirm which issues truly impact speed.
By connecting these layers, the system will avoid false alarms and will provide only confirmed, actionable recommendations. Its target users are software developers and data analysts who work with SQL Server but may lack deep tuning expertise.
Each key goal maps directly to measurable outcomes:

- **Goal 1:** Detect known inefficiencies → **Feature:** static analyzer → **Test:** unit test suite on query patterns → **Evidence:** `/artifacts/static_test_log.txt`
- **Goal 2:** Parse and interpret execution plans → **Feature:** runtime parser → **Test:** XML parsing and plan recognition cases → **Evidence:** `/artifacts/plan_parser_results.json`
- **Goal 3:** Correlate results and generate reports → **Feature:** combined analyzer → **Test:** integration and accuracy checks → **Evidence:** `/artifacts/report_samples.html`

System Architecture and Design

The architecture will consist of five layers:

1. **Database Access Layer** – Manages secure connection to SQL Server via `pyodbc`.
2. **Parser Layer** – Uses ANTLR with T-SQL grammar to convert SQL text into abstract syntax trees.
3. **Plan Analyzer Layer** – Reads and interprets XML execution plans for runtime cost data.
4. **Correlation Layer** – Maps findings from static and runtime analyses to determine confirmed inefficiencies.
5. **Reporting Layer** – Generates human-readable output in HTML and plain text.

Data flow will be linear: query input → parsing → plan retrieval → correlation → reporting. Each stage will log intermediate artifacts for transparency. The design will support safe failure—if runtime analysis fails, static results still generate a report.

Execution Plan Coverage Scope

SQL Server supports a very wide range of execution plan operators, but only a subset meaningfully affects performance in the OLTP-style workloads that QueryLens is designed to analyze. To maintain feasibility and ensure accuracy within the approved 360-hour development window, the system will focus on the most common and highest-impact operator types. The table below specifies exactly which operators and plan elements will be supported in this prototype, which will not be, and the rationale for each choice.

Execution Plan Component	Included?	Rationale
Index Seek / Index Scan	Yes	Essential for determining whether indexes are used efficiently.
Table Scan	Yes	Critical indicator of performance risk and missing indexes.

Key Lookup	Yes	Common cause of hidden performance slowdowns.
Nested Loop Join	Yes	Frequently used join type in OLTP workloads.
Hash Join	Yes	Often signals missing or ineffective indexes.
Merge Join	Yes	Important for comparisons involving sorted data.
Filter / Compute Scalar	Yes	Used to detect non-sargable predicates and implicit conversions.
Sort / Top / Stream Aggregate	Yes	Indicates memory-intensive operations that can affect performance.
Parallelism Operators (Distribute, Repartition, Gather Streams)	Yes	Necessary to detect unwanted or excessive parallel execution.
Cardinality Estimates	Yes	Required for correlating static findings with estimated runtime cost.
Actual Row Counts / Actual Executions	Yes	Confirms true runtime behavior during validation.

Columnstore Operators	No	Not used in the target POS OLTP environment; out of scope.
Batch Mode Operators	No	Relevant only for DW/BI systems; excluded to maintain focus.
Remote Query / Linked Server / PolyBase	No	Not present in the project's environment and would add unnecessary complexity.
CLR, External Script, or UDF Execution	No	Rare and outside the safety and scope boundaries of this prototype.
Distributed Query Operators	No	Not part of the single-server architecture approved for this project.

This coverage accounts for more than 95% of the execution-plan patterns encountered in the SQL Server workloads at my workplace, and therefore represents the most meaningful subset for practical performance analysis. Any unsupported operators will be detected gracefully and reported as neutral elements in the analysis pipeline. The selected subset provides full coverage of the performance characteristics necessary for accurate noise-free analysis without expanding the project beyond feasible limits.

Scope

In Scope: The prototype will focus exclusively on Microsoft SQL Server, performing:

- SQL Server query and stored procedure analysis.
- Static pattern detection (SELECT *, non-sargable conditions, implicit conversions).
- Execution plan parsing (XML extraction, cost estimation, index usage).
- Correlation between code and runtime findings.
- Report generation summarizing detected issues and recommendations.

Out of Scope:

- Support for non-SQL Server databases.
- Automated query rewriting or code deployment.
- Integration with third-party performance dashboards.
- Integration with CI/CD pipelines

Although automated query rewriting and CI/CD pipeline integration are excluded from the guaranteed deliverables of this project, QueryLens is intentionally structured so that these capabilities could be explored later. The use of ANTLR to produce an abstract syntax tree and the structured diagnostic outputs provide a natural foundation for optional, experimental query-suggestion or rewrite modules if time permits. These enhancements are not required for project completion but remain possible extensions should the core system be completed ahead of schedule.

Success Metrics

Success will be measured using the following quantifiable metrics and recorded artifacts.

Metric	Threshold	How Measured	Artifact or Log Location
Detection Accuracy	$\geq 90\%$ of known inefficiencies correctly flagged	30 query test cases with expected flags	/artifacts/detection_results.json
False Positive Rate	$\leq 10\%$ of flagged issues	Manual validation sample	/artifacts/validation_log.txt
Execution Plan Correlation	$\geq 85\%$ correct matching between static and runtime issues	XML plan cross-validation tests	/artifacts/correlation_matrix.csv
Report Clarity	$\geq 4 / 5$ average on usability survey	Feedback from 5 peer reviewers	/artifacts/usability_summary.csv
Performance Overhead	$\leq 5\%$ slower than baseline script execution	Time trials across test suite	/artifacts/performance_log.csv

Innovation and Differentiation

QueryLens is distinctive because it merges static and runtime SQL analysis to produce actionable optimization advice — a feature rarely integrated in lightweight developer tools. Conventional solutions such as SQL Server Management Studio's built-in suggestions or standalone linting tools perform only one form of analysis: either syntax-based or plan-based. QueryLens is designed to unify both and will rank issues using real execution evidence, thereby increasing diagnostic precision.

Compared to these baselines, QueryLens is expected to achieve a measurable edge: in validation tests, its combined analysis is expected to reduce false positives by at least 20% relative to static-only tools and identify approximately 30% more actionable optimizations within the same runtime constraints. All improvements are projected to be achieved while maintaining the approved limits — single server footprint, 24-hour data retention, and 10-second batch latency.

This measurable distinctiveness is demonstrated directly through artifacts in the evaluation plan: [artifacts/comparison_summary.csv](#) and [artifacts/combined_vs_baseline.png](#). By quantifying performance gains under identical conditions, QueryLens will allow its value to be verified objectively within the approved constraints.

Risks and Mitigations

1. **SQL Complexity Risk:** Not all SQL features are supported initially → Focus on common query patterns and gradually extend grammar coverage.
2. **Version Compatibility Risk:** Execution plan XML structures vary by SQL Server version → Test across multiple releases early in development.
3. **Correlation Accuracy Risk:** Linking code and runtime elements incorrectly → Implement incremental validation with known test queries.
4. **Usability Risk:** Reports may be too technical for non-expert users → Use plain language with short examples and definitions.

Testing & Validation Plans

Testing will verify that QueryLens meets its functional, performance, and usability goals through approximately **30 structured test cases** divided across three categories: *static analysis accuracy*, *execution plan interpretation*, and *report generation integrity*. Each case corresponds to a reproducible SQL input stored in the project's `/datasets/test_queries.sql` file and produces evaluation logs in `/artifacts/eval/`.

Test Categories and Coverage

1. Static Analysis (10 cases)

Each case focuses on detecting a specific inefficiency pattern such as:

- a. Unnecessary column retrieval (`SELECT *`)
- b. Non-sargable predicates (`WHERE YEAR(date) = 2023`)
- c. Implicit type conversions
- d. Redundant subqueries
- e. Missing join conditions
- f. Overuse of temporary tables
- g. Unfiltered aggregations
- h. Nested queries with correlated dependencies
- i. Functions in WHERE clauses
- j. ORDER BY without index support

Expected Outcome: $\geq 90\%$ of these patterns correctly identified and labeled in `artifacts/static_eval.json`.

2. Execution Plan Correlation (10 cases)

Tests ensure that static warnings are validated against real execution plan data. Each query is executed in SQL Server using a test schema mirroring production structure. Execution plans are stored as XML files under `artifacts/plans/`.

Patterns tested include:

- a. Table scans versus index seeks
- b. Hash joins versus merge joins
- c. Missing index warnings
- d. Parallelism indicators
- e. Estimated versus actual row discrepancies

Expected Outcome: $\geq 85\%$ correlation between static findings and runtime evidence recorded in `artifacts/runtime_eval.json`.

3. Report Generation and Performance (10 cases)

Tests measure clarity, consistency, and latency of the generated reports. The report engine merges analysis results, prioritizes confirmed issues, and outputs summaries in HTML and JSON formats.

Scenarios include:

- a. Multi-query batch analysis (up to 50 queries)
- b. Duplicate issue filtering
- c. Severity ranking accuracy
- d. Response time under simulated load
- e. Error handling with malformed queries

Expected Outcome: Reports generated in ≤ 10 seconds per batch, with consistent recommendations in `artifacts/report_eval.json` and visual summaries in `artifacts/report.html`.

Schedule and Milestones

Each stage concludes with a verifiable artifact demonstrating measurable progress.

Stage	Dates	Duration	Outcome (Artifact)
Environment Setup	Week 1-2	20 hours	Configured development environment
Static Analysis Engine	Week 3-6	70 hours	/artifacts/analyzer/static_results.json
Execution Plan Parser	Week 7-10	60 hours	/artifacts/analyzer/plan_results.json
Correlation Engine	Weeks 11-14	60 hours	/artifacts/reports/correlation_summary.html
Reporting Interface	Weeks 15-18	40 hours	/artifacts/interface/report_output.html
Testing and Documentation	Weeks 19-22	40 hours	/artifacts/final/evaluation_report.pdf

Buffer & Review	Weeks 23-25	30 hours	/artifacts/final_submission/
-----------------	-------------	----------	------------------------------

Assumptions and Constraints

- Development occurs on a single Windows Workstation with local SQL Server Developer Edition
- All datasets are anonymized; no production data used.
- No network access or cloud dependencies during testing.
- Total compute limited to 16 GB RAM and one GPU (if required for parsing acceleration).

Ethics and Data Governance

All testing will use sanitized SQL data. No customer processing data will be stored. Generated reports will be retained only for internal evaluation and deleted within 30 days of project completion.

Feasibility and Build Plan

Development will occur in five phases totaling approximately 360 hours. Python's mature libraries, lightweight runtime, and compatibility with SQL Server through [pyodbc](#) will make it well suited for this task. Testing will use both local SQL Server Developer Edition and controlled datasets of stored procedures derived from anonymized examples. All data remains within a secure local environment—no customer data is used. The required hardware is a single workstation with 16 GB RAM and SQL Server installed.

Canonical Terms

Term	Definition
------	------------

Inefficiency Detection	The process of identifying patterns known to degrade SQL performance
Static Analysis	Examination of SQL text for inefficiencies without executing it
Execution Plan	XML document describing how SQL server processes a query
Non-Sargable Condition	A WHERE clause expression that prevents index use
Correlation Engine	Component linking static and runtime findings
Detection Accuracy	Percentage of known inefficiencies correctly identified by QueryLens
Runtime Correlation Accuracy	Percentage of static findings confirmed by execution plan
Diagnostic Report	Generated output summarizing issues and recommendations
False Positive Rate	Percentage of reported inefficiencies not confirmed in runtime tests

Related Work

Several commercial and open-source tools assist with SQL performance tuning, such as SQL Server Management Studio's native *Query Analyzer* and *SolarWinds Database Performance Analyzer*. However, these tools either require expert interpretation or lack integration between code analysis and execution plan validation. QueryLens distinguishes itself by merging these perspectives—automating both code inspection and runtime evaluation to provide context-aware, plain-language feedback.

References

Microsoft Docs: Execution Plans –
<https://learn.microsoft.com/en-us/sql/relational-databases/performance/display-an-actua>

I-execution-plan

SQL Server Query Performance Tuning, Grant Fritchey

ANTLR T-SQL Grammar – <https://github.com/antlr/grammars-v4/tree/master/sql/tsql>